

Hands-On Graph Neural Networks Using Python

Chapter 1–2 상세 정리

A7610 그래프신경망특론

March 11, 2026

Contents

1	Chapter 1: Getting Started with Graph Learning	2
1.1	왜 그래프인가? (Why graphs?)	2
1.2	왜 Graph Learning인가? (Why graph learning?)	2
1.3	왜 GNN인가? (Why GNN?)	3
2	Chapter 2: Graph Theory for Graph Neural Networks	4
2.1	그래프의 기본 성질	4
2.2	중요 그래프 유형	4
2.3	기초 객체: degree, neighbors, path	4
2.4	그래프 측도: Centrality와 Density	4
2.5	그래프 표현 방법	5
2.6	그래프 탐색 알고리즘: BFS와 DFS	5
2.7	BFS vs DFS 요약 비교	6
3	시험/발표 포인트 정리	7

1 Chapter 1: Getting Started with Graph Learning

1.1 왜 그래프인가? (Why graphs?)

그래프는 객체와 객체 사이의 관계를 1급 정보로 다룬다. Tabular 데이터는 객체의 속성(attribute)은 잘 담지만, 객체 간 연결 구조를 직접적으로 표현하기 어렵다.

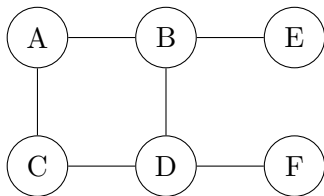


Figure 1: 관계 구조를 직접 표현하는 그래프 예시

그래프가 자연스러운 도메인

- 소셜 네트워크: 사용자(노드)-친구관계(엣지)
- 분자 구조: 원자(노드)-결합(엣지)
- 교통망: 교차로(노드)-도로(엣지)
- 추천 시스템: 사용자/아이템 이분그래프

그래프의 장점과 난점

- 장점: 고정 격자(이미지)나 순차(텍스트)로 담기 어려운 비정형 관계를 표현
- 난점: 노드 수/연결 수가 샘플마다 다르고 순서 불변성(permutation invariance)을 고려해야 함

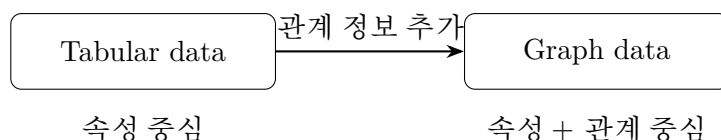


Figure 2: 테이블 표현에서 그래프 표현으로의 관점 전환

1.2 왜 Graph Learning인가? (Why graph learning?)

그래프 학습은 그래프 구조와 노드/엣지 특징을 함께 사용해 예측 문제를 푼다.

대표 과제

1. Node Classification: 노드 라벨 예측
2. Link Prediction: 누락/미래 엣지 예측
3. Graph Classification: 그래프 단위 라벨 예측
4. Graph Generation: 조건을 만족하는 신규 그래프 생성

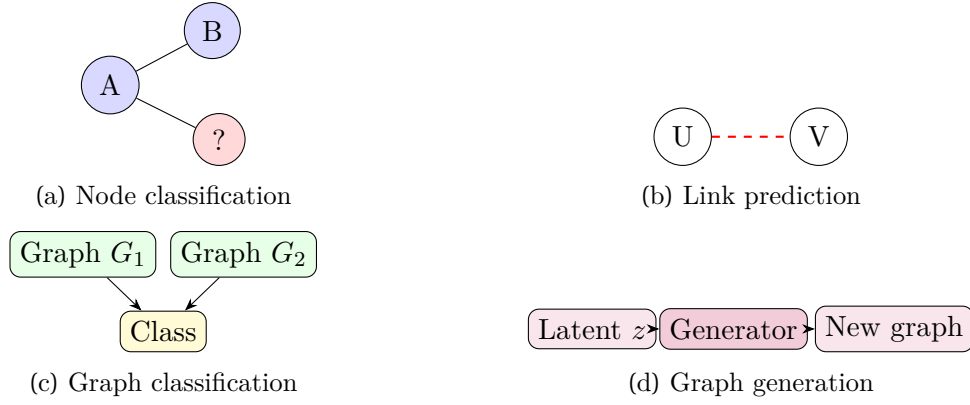


Figure 3: 그래프 학습의 핵심 과제들

그래프 학습 기법군(책 기준)

- Graph signal processing
- Matrix factorization
- Random walk
- Deep learning (GNN)

1.3 왜 GNN인가? (Why GNN?)

GNN은 메시지 패싱(message passing)으로 이웃 정보를 반복 집계하여 노드 표현을 업데이트 한다.

$$h_v^{(k)} = \phi^{(k)} \left(h_v^{(k-1)}, \bigoplus_{u \in \mathcal{N}(v)} \psi^{(k)}(h_v^{(k-1)}, h_u^{(k-1)}, e_{uv}) \right)$$

여기서 $\bigoplus$ 는 합/평균/최대 등 집계 연산이다.

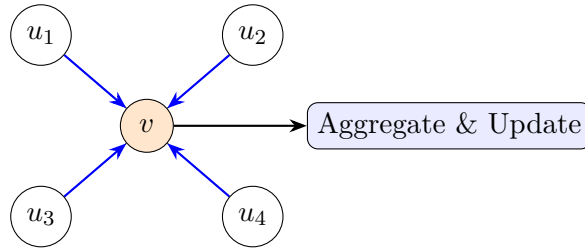


Figure 4: 메시지 패싱: 이웃 정보를 모아 중심 노드 표현을 갱신

GNN이 강한 조건

- 문제 복잡도가 높고 관계 구조가 예측 성능에 큰 영향을 줄 때
- 대규모 데이터에서 표현학습(representation learning)이 필요할 때

주의점

- 과도한 레이어 깊이는 over-smoothing을 유발할 수 있음
- 소규모 데이터/단순 규칙 문제에서는 비딥러닝 기법이 효율적일 수 있음

2 Chapter 2: Graph Theory for Graph Neural Networks

2.1 그래프의 기본 성질

그래프는 보통 $G = (V, E)$ 로 표기한다. V 는 노드 집합, E 는 엣지 집합이다.



Figure 5: 방향성 여부



Figure 6: 가중치 여부



Figure 7: 연결성 여부

2.2 중요 그래프 유형

2.3 기초 객체: degree, neighbors, path

- 무방향 degree: $\deg(v)$
- 유방향 indegree/outdegree: $\deg^-(v), \deg^+(v)$
- path: 엣지들의 연쇄, cycle: 시작과 끝이 같은 path

2.4 그래프 측도: Centrality와 Density

Centrality

- Degree centrality: 연결 수 기반 중요도
- Closeness centrality: 다른 노드까지 평균 최단거리 기반 접근성
- Betweenness centrality: 최단경로 사이의 중개자 역할

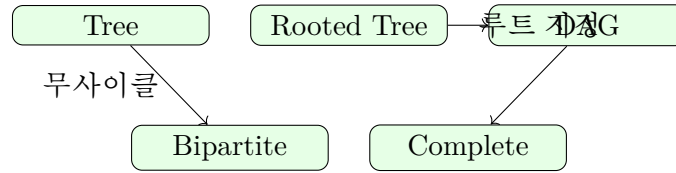


Figure 8: 자주 등장하는 그래프 유형 개요

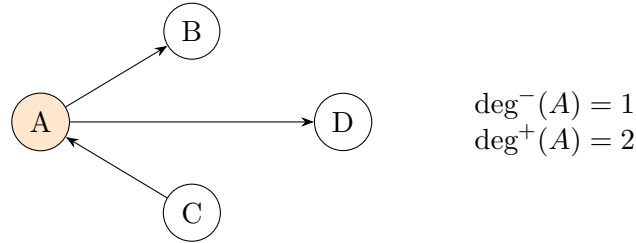


Figure 9: 유방향 그래프에서의 indegree/outdegree 예시

Density

$$\rho_{\text{undir}} = \frac{|E|}{|V|(|V|-1)/2}, \quad \rho_{\text{dir}} = \frac{|E|}{|V|(|V|-1)}$$

밀도는 연결의 조밀함을 나타내며, 일반적으로 낮으면 희소(sparse), 높으면 조밀(dense) 그래프로 본다.

2.5 그래프 표현 방법

표현 방식	장점	단점
Adjacency Matrix	엣지 존재 여부 확인이 $O(1)$, 행렬 연산 활용 용이	공간복잡도 $O(V ^2)$ 로 대규모 희소 그래프에서 비효율
Edge List	저장이 단순, 희소 그래프 저장 효율 높음	엣지 조회 시 리스트 탐색이 필요해 느릴 수 있음
Adjacency List	공간복잡도 $O(V + E)$, 이웃 순회 효율적	엣지 존재 확인은 평균적으로 Matrix보다 느림

Table 1: 그래프 표현 방식 트레이드오프

2.6 그래프 탐색 알고리즘: BFS와 DFS

BFS (Breadth-First Search)

- 큐를 이용해 레벨 순서로 탐색
- 무가중치 그래프 최단거리 탐색에 유리
- 시간복잡도: $O(|V| + |E|)$

DFS (Depth-First Search)

- 스택/재귀로 한 경로를 깊게 탐색 후 백트래킹

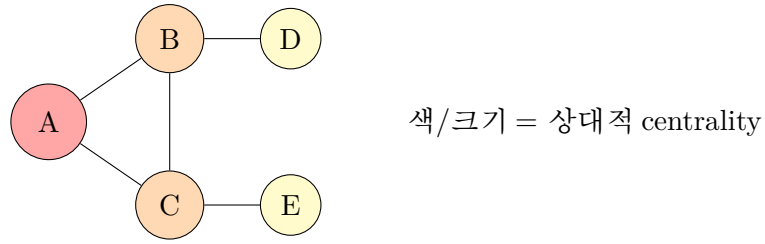


Figure 10: 중심성 직관: 중심부 노드가 정보 흐름에 더 큰 영향

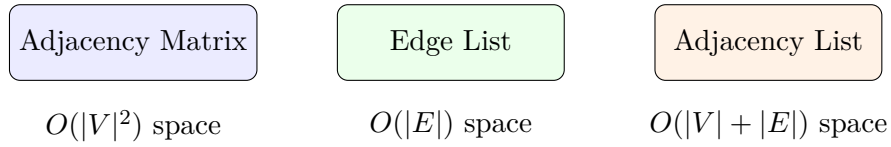


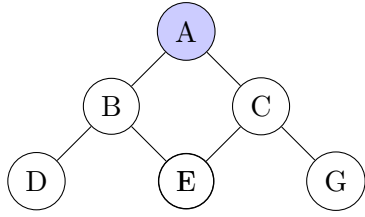
Figure 11: 세 가지 대표 그래프 저장 방식

- 사이클 탐지, 위상정렬, connected component 탐색 기반
- 시간복잡도: $O(|V| + |E|)$

2.7 BFS vs DFS 요약 비교

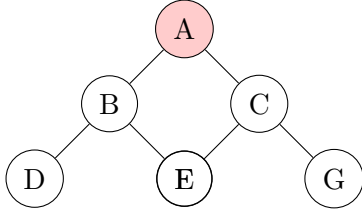
항목	BFS	DFS
핵심 자료구조	Queue	Stack / Recursion
탐색 특징	가까운 노드부터 확장	한 분기를 깊게 탐색
강점	무가중치 최단경로	구조 탐색/사이클 관련 문제
약점	넓은 그래프에서 메모리 부담	최단경로 보장 안 됨

Table 2: 두 탐색 알고리즘의 실전 선택 기준



BFS order: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Figure 12: BFS: 레벨 단위 확장



DFS order (example): $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

Figure 13: DFS: 깊이 우선 탐색 후 백트래킹

3 시험/발표 포인트 정리

1. “왜 그래프인가”를 속성+관계 정보 결합 관점에서 설명할 것
2. 그래프 학습 4대 과제(Node/Link/Graph/Generation)를 사례와 함께 구분할 것
3. GNN의 핵심이 이웃 집계(message passing)임을 수식과 직관으로 설명할 것
4. 방향성/가중치/연결성과 특수 그래프(Tree, DAG, Bipartite)를 명확히 구분할 것
5. 중앙성/밀도/표현방식(Matrix/List)의 트레이드오프를 비교할 것
6. BFS/DFS의 선택 기준을 문제 유형(최단거리 vs 구조탐색)에 연결할 것